Smart Student Schedule Builder

Pattern programming project

Instructor: Dr. Issam Jebreen

The Project is Presented By:

Mohammad Ibrahim Nayef Abu Salim	202301824 (Team Leader)
Mustafa Maher Al-Mashini	202300993
Hamza Jawdat Sarsak	202300587
Qusay Asim Al-Attari	202304136

Abstract :

The Smart Schedule Builder System assists university students in constructing their academic schedules in a structured and flexible manner. It allows users to select courses, assign lecture and lab times, and ensures that no scheduling conflicts or capacity violations occur. By applying the Builder Design Pattern, the system simplifies the construction of complex schedules while improving flexibility, scalability, and maintainability.

Introduction and Problem Statement :

University course registration systems are inherently complex due to the wide variety of available courses, multiple lecture and lab timings, and limited capacity for each class.

Students often face several challenges, including:

- Difficulty selecting suitable lecture and lab times
- Conflicts between different courses
- Limited seats in sections (capacity constraints)
- Lack of flexibility in traditional scheduling systems

Traditional approaches rely on rigid logic such as conditional statements, making the system difficult to maintain and extend. Therefore, a more flexible and structured approach is required to efficiently construct valid schedules.

Pattern Comparison – Summary:

Two patterns were considered: **Builder** and **Factory**.

Builder Pattern (Selected):

- Constructs objects step-by-step
- High flexibility and customization
- Checks constraints (time conflicts, capacity)
- Slightly more complex design

Factory Pattern (Alternative):

- Creates objects in a single step
- Simple to implement
- Limited customization
- Not suitable for schedules with optional parts

Conclusion:

The **Builder Pattern** is chosen because student schedules require incremental construction with validations, making it more flexible and maintainable than the Factory Pattern.

Project Aim and Objectives

Aim:

To develop a flexible and efficient system that allows students to build their schedules step by step without conflicts.

Objectives:

- Enable students to select courses
- Allow selection of lecture and lab times
- Detect and prevent scheduling conflicts
- Enforce capacity constraints for each section
- Provide a clear and structured final schedule
- Apply the Builder Design Pattern to improve system design

Proposed Solution

The proposed solution is a **Smart Schedule Builder System** that utilizes the Builder Design Pattern to construct student schedules step by step.

Instead of building the schedule in a single complex process, the system divides it into smaller steps:

- Selecting courses
- Assigning lecture times
- Assigning lab sessions
- Validating constraints (conflicts and capacity)

This approach ensures a flexible and maintainable system that can easily adapt to future changes.

Methodology and Application

The system is implemented using the **Builder Design Pattern**, which separates the construction of a complex object (Schedule) from its representation.

Application Flow:

1. The student logs into the system.
2. The student selects courses step by step.
3. The builder assigns lecture times.
4. The builder assigns lab sessions (if required).
5. The system checks for:
    - Time conflicts
    - Capacity constraints
6. The final schedule is generated and displayed.

This methodology ensures step-by-step construction, improving flexibility, scalability, and maintainability.

Stakeholder Analysis and Identification:

Stakeholders are individuals or groups that have an interest in or are affected by the system.

• Students:

Description: Primary users of the system
Role: Build and customize their schedules step-by-step
Interest: High
Influence: Medium

• Instructors:

Description: Teaching staff involved in course delivery
Role: Monitor class enrollment and manage teaching schedules
Interest: Medium
Influence: Medium

• Admin (Registrar Office):

Description: University administrative staff
Role: Manage courses, schedules, and capacity constraints
Interest: High
Influence: High

• System Developers:

Description: Development team building the system
Role: Implement, test, and maintain the application
Interest: High
Influence: High

• Instructor (Evaluator):

Description: Course instructor supervising the project
Role: Evaluates system design and implementation
Interest: High
Influence: Medium

.

User Stories:

Student:

- As a Student, I want to log in to the system, so that I can access the smart schedule builder dashboard.
- As a Student, I want to select my courses step-by-step, so that I can build a customized schedule that fits my preferences.
- As a Student, I want to choose lecture and lab times, so that I can organize my schedule effectively.
- As a Student, I want the system to check for time conflicts, so that I can ensure my lectures and labs do not overlap.
- As a Student, I want the system to check capacity constraints, so that I can enroll only in available sections.
- As a Student, I want to modify my schedule during the building process, so that I can adjust my selections easily.

- As a Student, I want to view my final schedule before confirmation, so that I can ensure it meets my needs.

Instructor:

- As an Instructor, I want to monitor the enrollment capacity for my classes, so that I can manage the learning environment effectively.
- As an Instructor, I want to view the schedule of my assigned classes, so that I can organize my teaching plan.

Admin (Registrar):

- As an Admin, I want to set capacity constraints for each section, so that the system can automatically prevent over-enrollment.
- As an Admin, I want to manage course offerings and schedules, so that students have updated options.
- As an Admin, I want to control available time slots, so that scheduling conflicts are minimized.

Functional and Non-Functional Requirements:

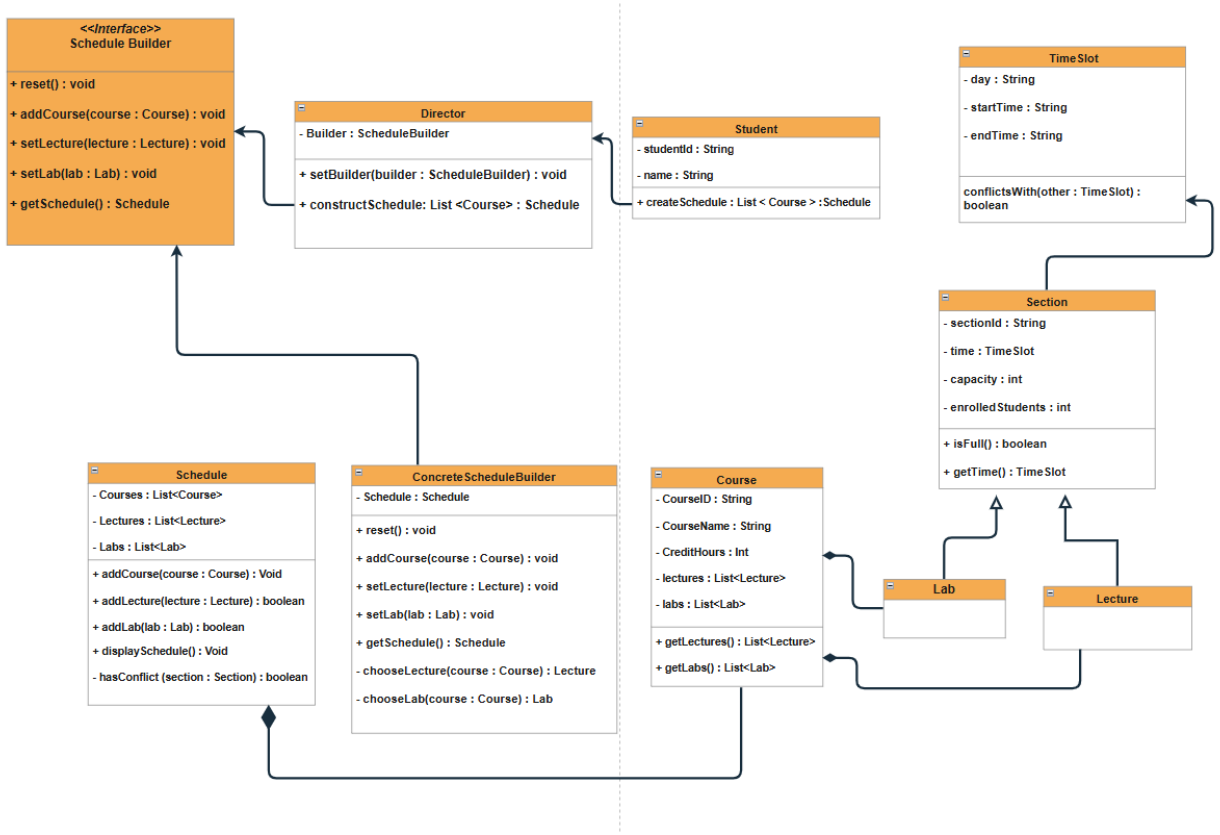
Functional Requirements:

- The system allows students to log in.
- The system allows course selection.
- The system allows choosing lecture and lab times.
- The system detects scheduling conflicts.
- The system checks capacity constraints.
- The system allows modifying the schedule.
- The system builds and displays the final schedule.

Non-Functional Requirements:

- The system should be scalable.
- The system should be maintainable.
- The system should provide fast response time.
- The system should be user-friendly.
- The system should be reliable and accurate.

Class Diagram:



Class Description Template:

UML Class Documentation -Smart Student Schedule Builder

This section presents the main classes of the system in textual format, including their attributes, operations, and relationships based on the UML class diagram.

Student:

Class Name: Student

Description: Represents a university student who initiates the process of creating an academic schedule.

Attributes:

- studentId : String
- name : String

Operations:

- createSchedule(courses : List<Course>) : Schedule

Relationships:

- Associated with Director (uses it to build the schedule)

Director:

Class Name:
Director

Description:
Controls the construction process of the schedule by coordinating with the ScheduleBuilder.

Attributes:

- builder : ScheduleBuilder

Operations:

- setBuilder(builder : ScheduleBuilder) : void
- constructSchedule(courses : List<Course>) : Schedule

Relationships:

- Associated with ScheduleBuilder

---

3. ScheduleBuilder:

Class Name: ScheduleBuilder (Interface)

Description: Defines the abstract steps required to build a schedule incrementally.

Attributes: None

Operations:

- reset() : void
- addCourse(course : Course) : void
- setLecture(lecture : Lecture) : void
- setLab(lab : Lab) : void
- getSchedule() : Schedule

Relationships:

- Implemented by ConcreteScheduleBuilder

---

4. ConcreteScheduleBuilder:

Class Name: ConcreteScheduleBuilder

Description: Implements the ScheduleBuilder interface and provides the actual logic for constructing and validating the schedule.

Attributes:

- schedule : Schedule

Operations:

- reset() : void
- addCourse(course : Course) : void
- setLecture(lecture : Lecture) : void
- setLab(lab : Lab) : void
- getSchedule() : Schedule
- chooseLecture(course : Course) : Lecture (private)
- chooseLab(course : Course) : Lab (private)

Relationships:

- Implements ScheduleBuilder
- Composes Schedule
- Uses Course, Lecture, and Lab

---

5. Schedule:

Class Name: Schedule

Description: Represents the final schedule containing selected courses, lectures, and lab sessions with validation for conflicts.

Attributes:

- courses : List<Course>
- lectures : List<Lecture>
- labs : List<Lab>

Operations:

- addCourse(course : Course) : void
- addLecture(lecture : Lecture) : boolean
- addLab(lab : Lab) : boolean
- displaySchedule() : void
- hasConflict(section : Section) : boolean

Relationships:

- Composes Course
- Composes Lecture
- Composes Lab

---

6. Course:

Class Name: Course

Description: Represents a university course that contains multiple lecture and lab options.

Attributes:

- courseId : String
- courseName : String
- creditHours : int
- lectures : List<Lecture>
- labs : List<Lab>

Operations:

- getLectures() : List<Lecture>
- getLabs() : List<Lab>

Relationships:

- Composes Lecture
- Composes Lab

7. Section:

Class Name: Section

Description: An abstract representation of a course section (lecture or lab) that includes scheduling and capacity information.

Attributes:

- sectionId : String
- time : TimeSlot
- capacity : int
- enrolledStudents : int

Operations:

- isFull() : boolean
- getTime() : TimeSlot

Relationships:

- Associated with TimeSlot
- Parent of Lecture and Lab

---

8. Lecture:

Class Name: Lecture

Description: Represents a lecture section of a course. It inherits all properties from Section.

Attributes:

- Inherits from Section

Relationships:

- Inherits from Section

---

9. Lab:

Class Name: Lab

Description: Represents a lab section of a course. It inherits all properties from Section.

Attributes:

- Inherits from Section

Relationships:

- Inherits from Section

---

10. TimeSlot :

Class Name: TimeSlot

Description: Represents the time details of a section, including day and start/end times, and provides conflict detection functionality.

Attributes:

- day : String
- startTime : String
- endTime : String

Operations:

- conflictsWith(other : TimeSlot) : boolean

Relationships:

- Associated with Section

Conclusion :

In conclusion, the Smart Student Schedule Builder provides an effective and structured solution to the common challenges students face during course registration, particularly time conflicts and limited section availability. By applying the Builder Design Pattern, the system enables the construction of schedules in a step-by-step manner, ensuring that all constraints are validated throughout the process rather than at the end.

The system demonstrates strong object-oriented design principles, including separation of concerns, modularity, and scalability. Each class has a clearly defined responsibility, which improves maintainability and allows for future extensions such as adding new validation rules or optimizing section selection strategies.

Furthermore, the use of abstraction through the ScheduleBuilder interface and its concrete implementation enhances flexibility, making it easier to modify or extend the scheduling logic without affecting other components.

Overall, the system improves the student registration experience by reducing errors, saving time, and generating reliable, conflict-free schedules. It also serves as a practical example of how design patterns can be effectively applied to solve real-world problems in software engineering.